Insurance Policy Approval: Combining Risk Probability and Potential Loss Prediction

Authors: Matthew Hawkes, Christopher Tsialas

Team: RiskGuard Group

Date: 12/17/25

Objective

The objective of this project is to design a program for insurance policy approval by jointly modeling claim probability and claim severity. To do this, we decompose insurance risk into two components: whether a claim will occur and, if it does, how costly that claim would be. This allows insurers to make more informed approval, rejection, or pricing decisions.

Executive Summary

To capture both metrics, claim probability and claim severity, we built two predictive models using real-world insurance datasets. A classification model estimates the probability that a policyholder files a claim, while a regression model estimates the expected loss amount for a claim.

Exploratory data analysis revealed strong class imbalance in the claims dataset and heavy right-skew in loss severity, forcing us to adjust our models to these conditions. After feature engineering, preprocessing, and model comparison, we tuned XGBoost models for both tasks, classification and regression. Our final metric combines both claim probability and expected loss, which can be used to inform insurance policy decisions.

Problem Framing and Datasets

Two datasets were used to predict insurance risk:

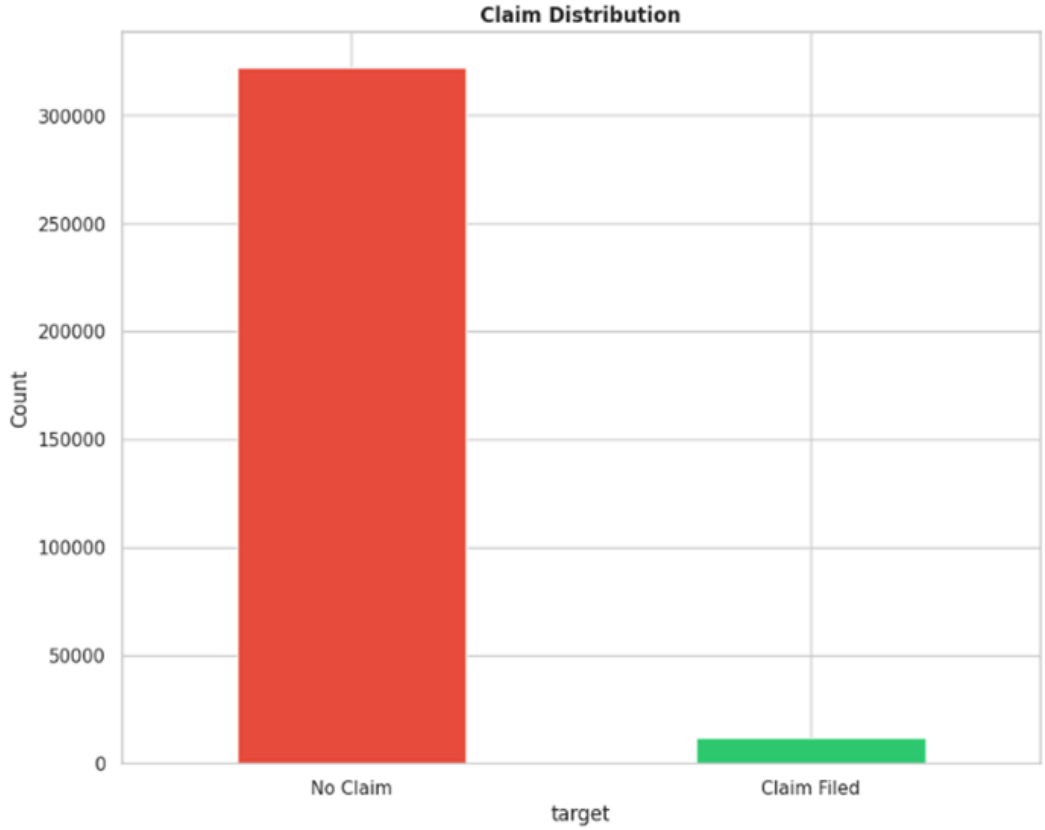
1. **Porto Seguro Safe Driver Prediction (Classification)**
    - Each observation represents a driver/policy profile.
    - Target variable is binary: claim filed (1) or not (0).
    - Key challenge: claims are rare, leading to severe class imbalance.
2. **Allstate Claims Severity (Regression)**
    - Each observation represents a claim case.
    - Target variable is continuous loss amount.
    - Key challenge: loss values are highly skewed with extreme outliers.

Success metrics were chosen to align with business risk, prioritizing recall and probability ranking for classification because of class imbalance, and magnitude-sensitive error metrics for regression because of extreme outliers.

Exploratory Data Analysis (EDA)

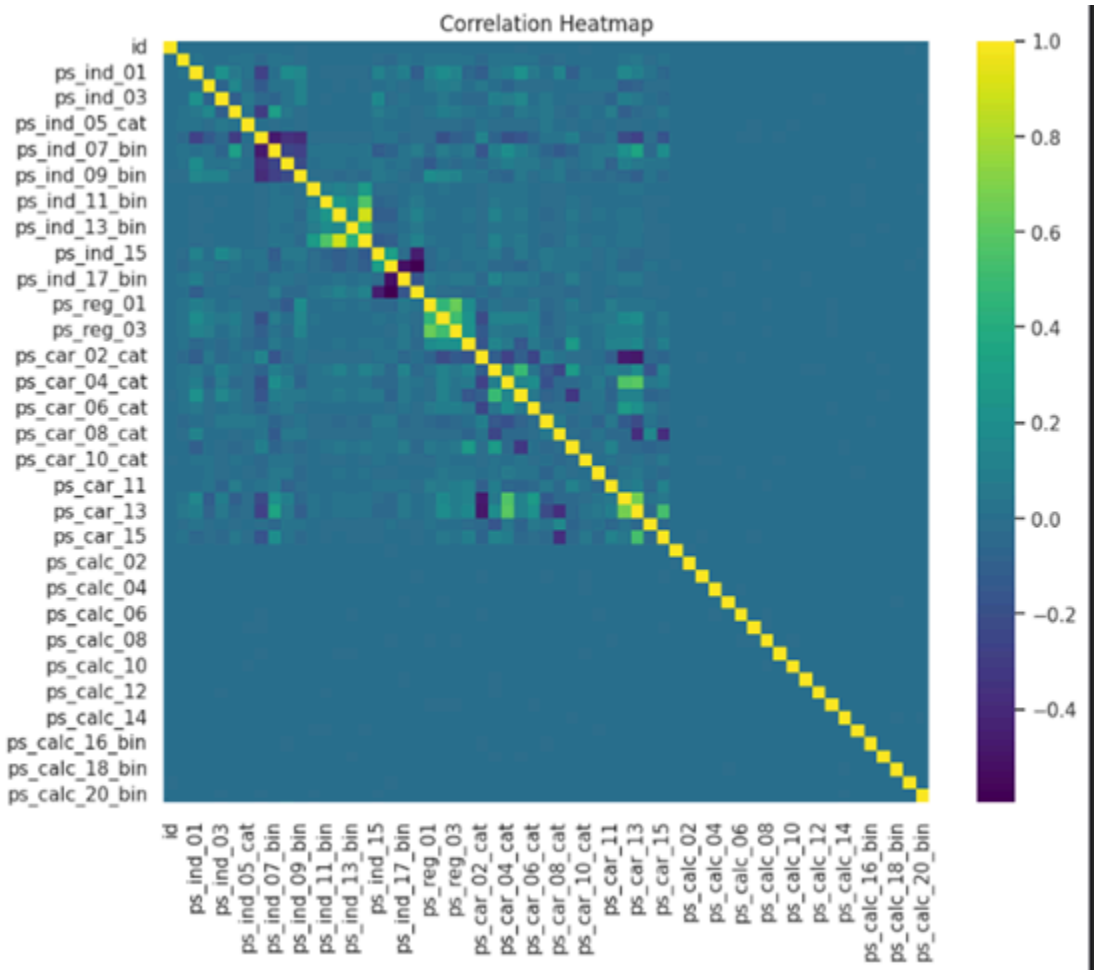
Classification Dataset

Claim Distribution



96.36% of the data consists of cases for no claim, while only 3.64% of the data consists of cases for claims. This shows that claims are rare, so the target is strongly imbalanced, which makes accuracy misleading.

Correlation Heatmap



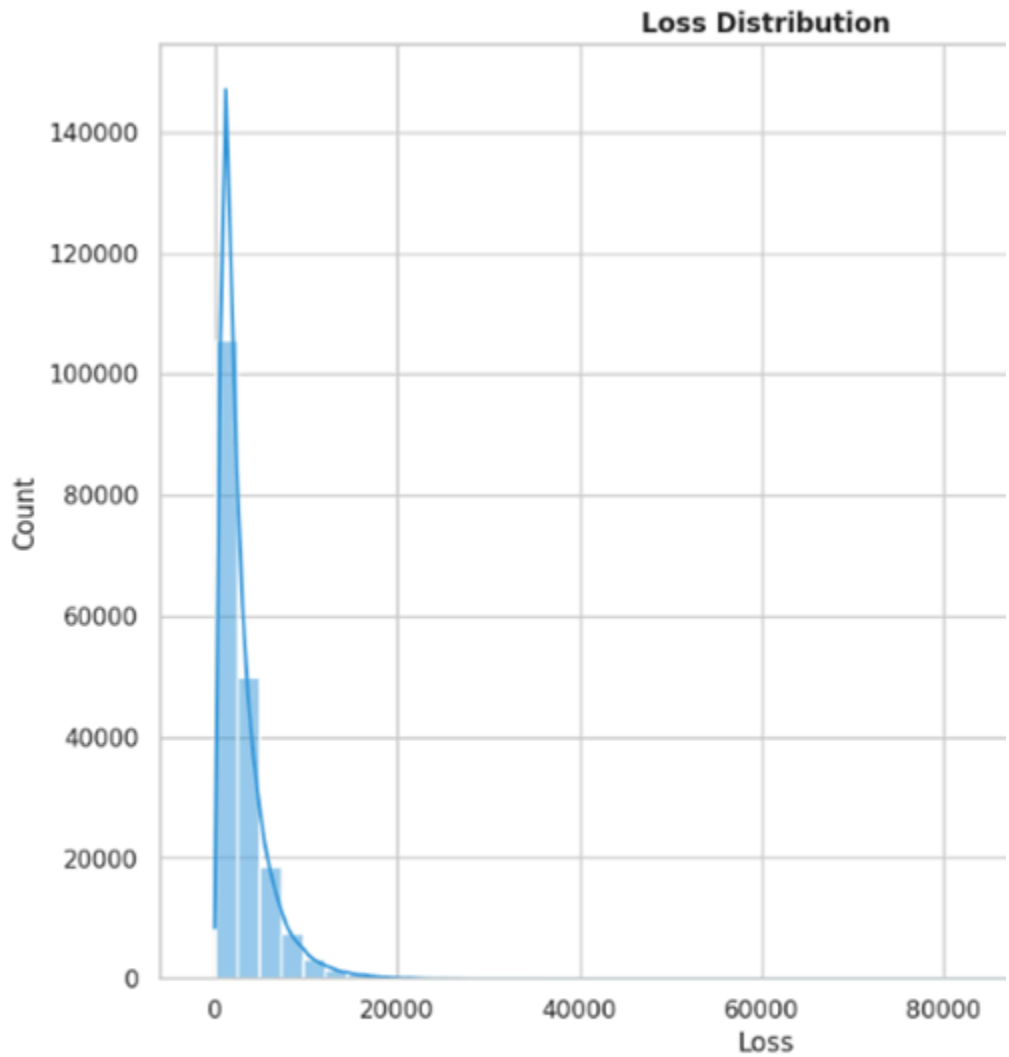
Correlation analysis showed weak individual feature relationships, suggesting signal is distributed across many variables. Not one obvious or single predictor suggests there is a claim or not a claim.

Missing Values

Values are missing at a rate lower than 0%, so in very few rows. Missingness is concentrated in a handful of columns, so targeted imputation makes more sense than dropping rows of data.

Regression Dataset

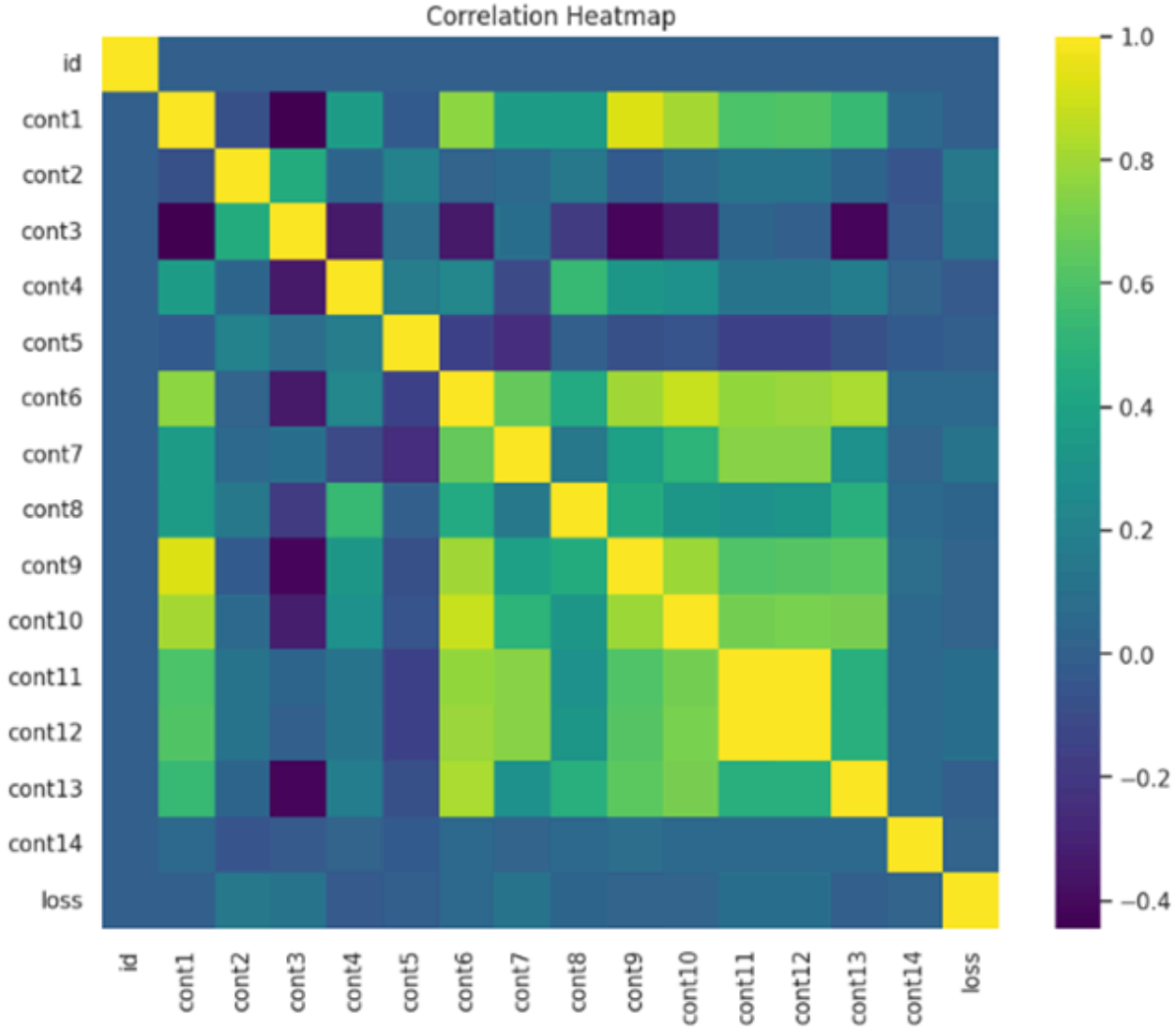
Loss Distribution




```
Loss summary:
count    188318.000000
mean       3037.337686
std        2904.086186
min           0.670000
25%        1204.460000
50%        2115.570000
75%        3864.045000
max      121012.250000
```

Loss distribution is heavily right-skewed, with a small number of large claims driving total risk. Mean loss is not representative, so MAE is critical for evaluating “typical” error. Large claims can dominate RMSE so track both MAE and RMSE. it is also tracked to understand sensitivity to large claims.

Correlation Heatmap



Most feature correlations are weak, motivating non-linear models in our project. Some non-linear classification models we could use are RandomForest and XGBoost.

Feature Engineering

Classification Feature Engineering

```
Binary Features by Signed Claim-Rate Difference (%):
ps_ind_10_bin             +2.24%
ps_ind_17_bin             +2.14%
ps_ind_13_bin             +1.74%
ps_ind_12_bin             +1.59%
ps_ind_07_bin             +1.46%
ps_ind_06_bin             -1.29%
ps_ind_11_bin             +1.19%
ps_ind_16_bin             -1.09%
ps_ind_08_bin             +0.67%
ps_ind_09_bin             -0.42%
ps_ind_18_bin             +0.22%
ps_calc_20_bin            -0.04%
ps_calc_16_bin            +0.04%
ps_calc_15_bin            -0.03%
ps_calc_18_bin            +0.03%
ps_calc_19_bin            -0.03%
ps_calc_17_bin            +0.00%
```

Computed the Claim-Rate Difference for each feature to measure how much it increases or decreases claim risk.

Positive values indicate increased risk; negative values indicate reduced risk.

This allows us to interpret feature impact even with anonymized feature names.

Regression Feature Engineering

```
Categorical Features by Signed Mean Loss Difference:
cat116                    48345.91
cat105                    48345.91
cat113                    36980.04
cat115                    30987.41
cat89                     15609.24
cat102                    15609.24
cat57                     7198.59
cat114                    6022.78
cat101                    5944.64
cat110                    5601.48
cat111                    5442.07
cat7                      5177.07
cat106                    5067.90
cat90                     4432.64
cat103                    4265.61
cat92                     4165.93
cat64                     3490.79
```

Computed Mean Loss Difference for each feature to estimate its impact on expected loss.

Captures how features shift claim severity on average, with positive values increasing loss and negative values decreasing loss.

RiskSum

```
Average RiskSum_total by Target:
Target 0: 0.0640
Target 1: 0.0860
```

```
Average RiskSum_total by Loss Quartile:
Quartile 0: 2886.9199
Quartile 1: 2923.4520
Quartile 2: 2929.2334
Quartile 3: 3064.6299
```

Combined signed differences across feature types into an aggregate RiskSum feature. It is a weighted sum (weighted on impact) of the presence of each feature.

Created simple rule-based flags (RiskFlag and HighRiskFlag) to provide interpretability and a binary feature for the ML models. Each feature is activated if the RiskSum reaches a certain threshold.

Preprocessing Pipeline

```python
def build_preprocessor(numeric_features, categorical_features):
    numeric_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ])

    categorical_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore'))
    ])

    return ColumnTransformer(
        transformers=[
            ('num', numeric_transformer, numeric_features),
            ('cat', categorical_transformer, categorical_features)
        ]
    )
```

Numeric features: median imputation and standardization.

Categorical features (claims dataset): most-frequent imputation and one-hot encoding.

Modeling and Evaluation

Classification Models Tested

- Logistic Regression
- K-Nearest Neighbors
- Random Forest
- XGBoost

	Model	Precision	Recall	F1-Score	ROC-AUC	Avg Precision	Balanced Accuracy
0	Logistic	0.0526	0.5464	0.0960	0.6233	0.0588	0.5870
1	KNN	0.0921	0.0016	0.0032	0.5127	0.0377	0.5005
2	RandomForest	0.0000	0.0000	0.0000	0.5755	0.0487	0.5000
3	XGBoost	0.0556	0.5024	0.1001	0.6293	0.0631	0.5898

Evaluation emphasized probability outputs, ROC AUC, and threshold selection that maximized precision. This is the best practice when facing a highly imbalanced dataset.

The massive, extremely imbalanced dataset made initial models not perform amazingly, but the data was still useful for determining the best model for final prediction.

Regression Models Tested

- Linear Regression
- Ridge Regression
- Random Forest Regressor
- XGBoost Regressor

	Model	RMSE	MAE	R2
3	XGBoost	3.431756e+06	1178.702677	0.579421
1	Ridge	4.012258e+06	1290.538441	0.508278
2	RandomForest	4.034452e+06	1290.559387	0.505558
0	Linear	4.066705e+06	1291.425777	0.501605

Models were evaluated using MAE, RMSE, and R^2 to determine the best option for our final model.

Final Model Selection

Classification

- Tuned XGBClassifier using RandomizedSearchCV, which provides good results while not undergoing exhaustive GridSearch.
- Also, we used StratifiedKFold, which ensures proportional imbalance in each fold from the dataset.
- Average_precision was our scoring metric as it better handles class imbalance than others, like accuracy.

Regression

- Tuned XGBRegressor using RandomizedSearchCV, which provides good results while not undergoing exhaustive GridSearch.
- For this, we used KFold, as StratifiedKFold is unnecessary for continuous data.
- Neg_root_mean_squared_error, which is better suited for continuous data.

Conclusion

Key Results

- A two-model approach better captures real insurance risk than a single score.
- XGBoost performed best across both tasks due to its ability to model non-linear relationships and handle weak individual signals.
- Combining outputs into expected cost provides a direct link to underwriting decisions.

Limitations

- Feature anonymization limits interpretability and domain-specific insights.
- Extreme class imbalance makes probability thresholds sensitive to small distribution shifts.
- Heavy-tailed loss distributions challenge traditional regression objectives.

Future Work

- Calibrate predicted probabilities and stress-test thresholds under changing claim rates.
- Explore log-transformed or robust loss functions for severity modeling.
- Incorporate cost-sensitive learning aligned with insurer risk tolerance.
- Deploy rule-based cutoffs on expected cost for real-world approve/reject/pricing policies.

Final Takeaway

Our models effectively predict the probability of a person filing a claim, and the potential loss of that claim. By explicitly modeling both of these targets, this framework transforms machine learning outputs into actionable insurance policy decisions where a company can approve, reject, or change their pricing.